

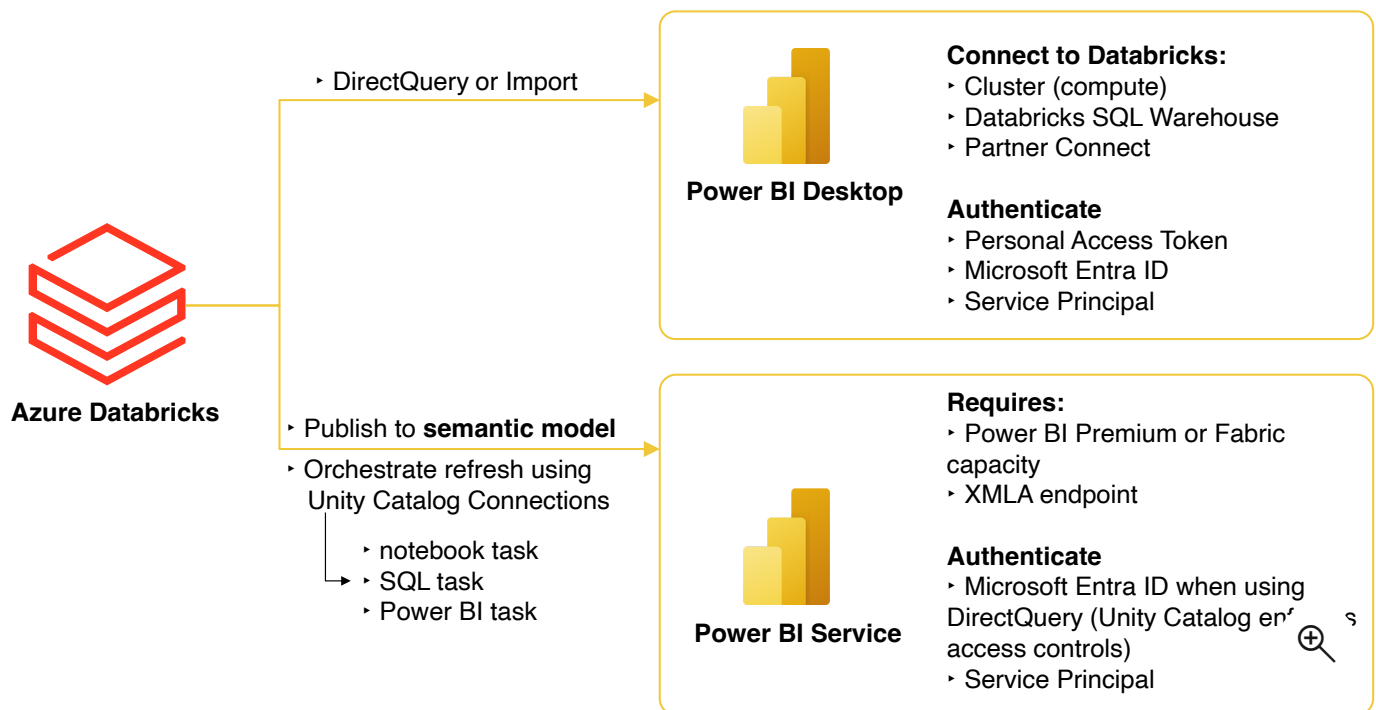


Understand integration with Power BI

12 minutes

Power BI provides interactive visualizations and business intelligence capabilities. Integrating Power BI with Azure Databricks allows users to create reports and dashboards from Databricks-managed data. This integration supports multiple user roles, from data engineers to business analysts.

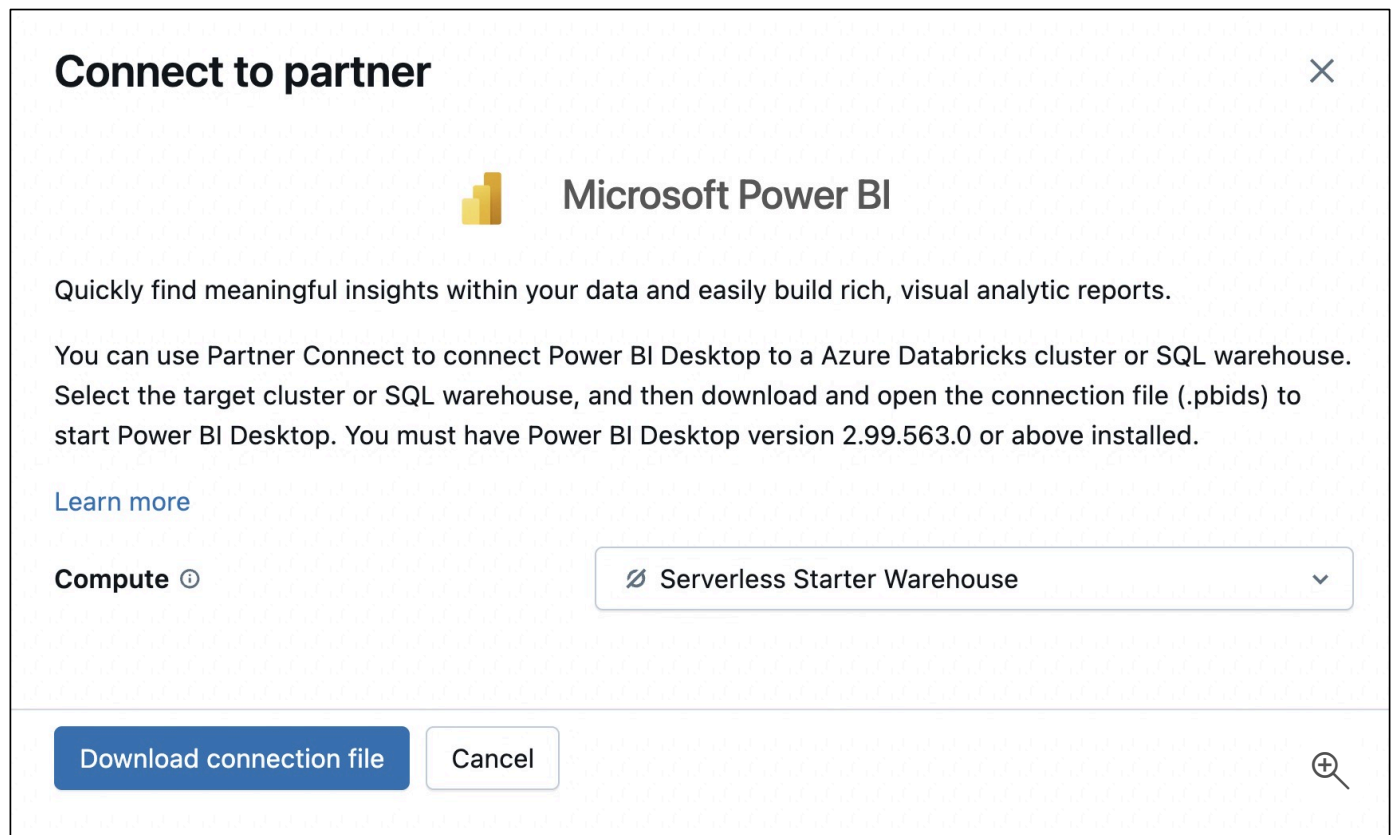
The integration between Azure Databricks and Power BI works through **multiple connection methods**. You can connect Power BI Desktop to your compute resources, publish data directly from Azure Databricks to the Power BI service, or orchestrate semantic model refreshes using Unity Catalog connections. Each method serves different scenarios and supports various authentication options.



Connect Power BI Desktop to Azure Databricks

Power BI Desktop is a Windows-based application that enables you to create interactive reports and dashboards. You can connect Power BI Desktop to both Azure Databricks **clusters** and **Databricks SQL warehouses**. Use **SQL warehouses with DirectQuery mode** for optimized query performance and built-in serverless scaling.

The fastest way to establish a connection is through **Partner Connect**. From the Azure Databricks workspace Marketplace, you select the Power BI tile, choose your compute resource, and download a connection file. This file opens Power BI Desktop with preconfigured connection settings. Alternatively, you can configure the connection manually by entering the **Server Hostname** and **HTTP Path** from your compute resource details.



After establishing the connection, you select your **data connectivity mode**. With **Import** mode, Power BI loads data into its internal storage engine, enabling fast queries but requiring periodic refreshes to stay current. With **DirectQuery** mode, Power BI queries Azure Databricks directly for each visualization, ensuring real-time data access but requiring an active connection and running compute resource.

ⓘ Note

Since February 2026, all new Power BI connections to Azure Databricks use the **Arrow Database Connectivity (ADBC)** driver by default. ADBC is a modern, columnar API standard based on Apache Arrow that offers more efficient data transfer for analytical workloads. Existing connections continue to use ODBC unless you manually update them by setting `Implementation="2.0"` in the Advanced Editor in Power BI Desktop. Test your existing semantic models against ADBC to verify compatibility before updating.

Authentication determines how Power BI accesses your data. **Personal access tokens** provide simple authentication but represent a single user identity. **Microsoft Entra ID** enables single sign-on where each user's credentials pass through to Azure Databricks, allowing Unity Catalog to enforce fine-grained permissions. For automated scenarios, **service principals** with machine-to-machine OAuth provide fixed credentials that don't depend on individual user accounts.

Power BI Desktop also supports **native SQL queries** for compute-intensive operations. Instead of using Power BI's visual query builder, you write SQL directly against your Databricks SQL warehouse. This approach gives you full control over query optimization and access to Databricks-specific SQL features.

ⓘ Note

Power BI Desktop requires Windows. If you use a different operating system, run Power BI Desktop on a Windows-based virtual machine and connect to it remotely.

Publish to the Power BI service from Azure Databricks

The Power BI service is a cloud-based platform where you share reports and collaborate with your organization. Rather than connecting Power BI to Databricks, you can publish data **from** Azure Databricks directly to the Power BI service. This workflow simplifies the publishing process by initiating it from the Azure Databricks UI.

To publish data, you use **Catalog Explorer** in your Azure Databricks workspace. You select a schema or specific tables from Unity Catalog, then choose **Publish to Power BI workspace**. The publishing wizard authenticates you with Microsoft Entra ID and prompts you to select your target Power BI workspace, choose between DirectQuery and Import modes, and configure authentication settings.

Connect to partner



Microsoft Power BI

Quickly find meaningful insights within your data and easily build rich, visual analytic reports.

You can use Partner Connect to directly publish to Power BI. Connect to Microsoft Entra ID to select a Power BI workspace and publish your data.

Power BI Premium is required for publishing data to your Power BI workspace.

[Learn more](#)

SQL warehouse ⓘ

Serverless Starter Warehouse

Power BI workspace* ⓘ

AdventureWorks ▼

Power BI semantic model*

☒ Publish new semantic model

☐ Update an existing semantic model

Select a semantic model ▼

Power BI query mode*

DirectQuery ▼

Tables to publish*

databricks_learn.adventure-works
24 out of 24 tables selected

▶ Select tables and set query modes

Authentication method in Power BI* ⓘ

OAuth ▼

Publish to Power BI

Cancel



This publishing method requires **Unity Catalog** for data governance and a **Power BI Premium license** (Premium capacity, Premium Per User, or Microsoft Fabric capacity). The Premium license provides the necessary capacity for enterprise scenarios and enables the XMLA (XML for Analysis) endpoint that Databricks uses to create and update semantic models programmatically.

When you publish a schema containing multiple tables, Databricks creates a **semantic model** in your Power BI workspace. **Column comments** from Unity Catalog tables transfer to Power BI as

column descriptions, preserving your documentation. **Foreign key relationships** also transfer, though Power BI supports only one active relationship path between any two tables, so some relationships might become inactive if multiple paths exist.

Use OAuth authentication for published semantic models to enable fine-grained access control and user-level auditing. When users access a report built on DirectQuery mode, Power BI passes their Microsoft Entra ID credentials to Azure Databricks. Unity Catalog evaluates permissions for that specific user, enforcing row-level filters, column masks, and table access controls. This approach maintains governance consistency across platforms.

For scenarios requiring automated refreshes or shared credentials, you can configure **machine-to-machine OAuth** using service principals. After publishing, you edit the semantic model's data source credentials in Power BI and provide the service principal's client ID and secret. This method works with both semantic models and Power BI gateways for private network scenarios.

Important

Publishing to Power BI service requires the XMLA endpoint to be enabled with Read and Write capabilities in your Power BI/Fabric capacity settings.

Automate Power BI publishing with Power BI tasks

While you can publish to Power BI manually from Azure Databricks, **Power BI tasks** enable you to orchestrate semantic model publishing and refreshes automatically as part of your Databricks workflows. This automation integrates Power BI updates into your data pipelines, ensuring that reports refresh only after upstream data transformations complete.

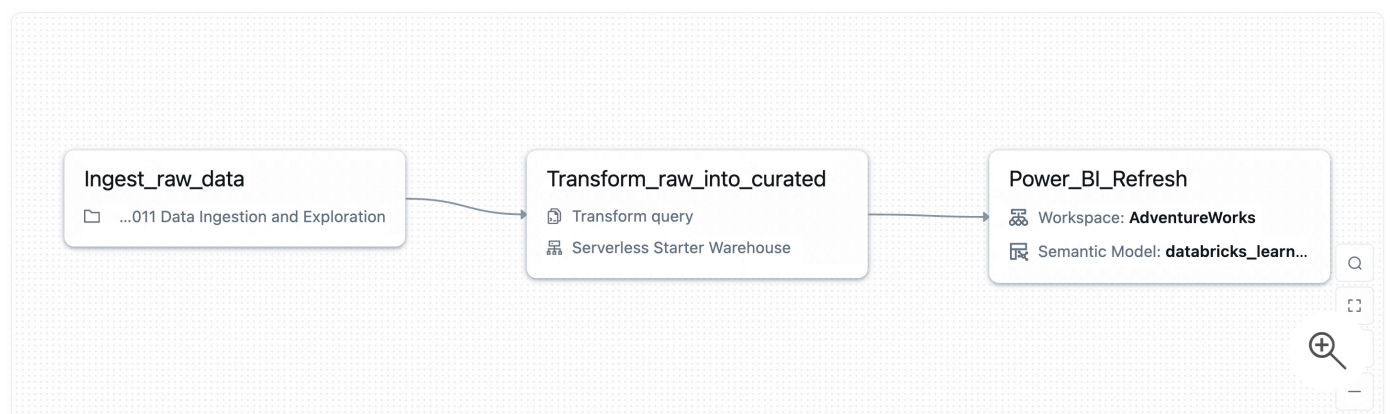
The screenshot displays the Databricks Jobs & Pipelines interface. On the left, a sidebar lists various workspace components. The main area shows the configuration for a 'Power BI Refresh Job'. The 'Type' is set to 'Power BI', and the 'SQL warehouse' is 'Serverless Starter Warehouse'. The 'Power BI connection' is 'powerbi', the 'Power BI workspace' is 'AdventureWorks', and the 'Power BI semantic model' is 'databricks_learn-adventure-works'. The 'Power BI query mode' is 'DirectQuery'. On the right, the 'Job details' panel shows the job ID '225462214182201', created by 'System Administrator', and run as 'System Administrator'. Below this, the 'Schedules & Triggers' section is visible. In the foreground, a workflow diagram illustrates the process: 'Ingest_raw_data' (Data Ingestion and Exploration) leads to 'Transform_raw_into_curated' (Transform query, Serverless Starter Warehouse), which then leads to 'Power_BI_Refresh' (Workspace: AdventureWorks, Semantic Model: databricks_learn...).

Power BI tasks require a **Unity Catalog connection** to Power BI. This connection stores authentication credentials and workspace information that the task uses to publish or update semantic models. You choose an authentication method (service credentials, OAuth machine-to-machine, or OAuth user-to-machine) based on your security requirements.

Power BI tasks integrate into Databricks workflows alongside data engineering tasks, machine learning model training, and other automation. You create task dependencies to control execution order, ensuring that Power BI semantic models refresh only after source data completes processing.

For example, a workflow might include:

- A **notebook task** that ingests raw data from external sources into Unity Catalog tables
- A **SQL task** that transforms the raw data into curated analytics tables
- A **Power BI task** that publishes the curated tables to a semantic model and triggers a refresh



When upstream data updates complete successfully, the Power BI semantic model automatically refreshes with the latest data.

Choose the right integration method

The choice between integration methods depends on your workflow and requirements. **Use Power BI Desktop connections** when business analysts need to explore data interactively and build reports. Desktop provides a report authoring environment with drag-and-drop visualizations and the flexibility to combine multiple data sources.

Use publishing from Azure Databricks to Power BI service when you want to share curated datasets with specific governance and want to initiate sharing from the data engineering side. This method ensures that data engineers control which tables become available in Power BI and can document them with column descriptions that transfer automatically.

Orchestrate Power BI updates with data pipelines when you need to automate semantic model refreshes as part of larger data pipelines. This approach coordinates Power BI updates with upstream data transformations, ensuring that reports refresh only after source data completes processing.

For organizations with private network requirements, Power BI gateways enable secure connectivity without exposing Azure Databricks endpoints publicly. You configure an on-premises or virtual network gateway with your Azure Databricks connection details and authentication credentials. Power BI service routes queries through the gateway, which connects to your private Azure Databricks workspace.

Security and governance considerations

Security architecture differs between connection methods and authentication types. The following table compares how Unity Catalog enforces permissions and tracks access:

 Expand table

Use case	Authentication method	Unity Catalog permission enforcement	Audit granularity
Interactive exploration with Power BI Desktop or DirectQuery reports requiring user-specific governance.	Microsoft Entra ID (SSO)	Evaluates permissions per user. Row-level filters, column masks, and table access controls apply based on each viewer's identity.	User-level audit logs show which specific users accessed which data.
Scenarios where all users should see identical data, or when implementing row-level security within Power BI.	Import mode or no SSO	Uses fixed credentials. All report viewers see the same data regardless of their Unity Catalog permissions.	Limited to the credential owner's identity. Can't track individual viewer access.
Automated workflows, scheduled refreshes, and scenarios where consistent data access is required.	Service principals	Evaluates permissions based on the service principal's grants, not individual viewers.	Shows only that the service principal accessed data, not which report viewers consumed it.
Development and testing.	Personal access tokens	Represents a single user identity. Permissions apply to that user only. Not recommended for production automation.	Shows the token owner accessed data, not the actual end user.

Next unit: Understand integration with VS Code

[< Previous](#)
[Next >](#)